

Data Structures

(Course Code: 25B11CI318)

Binomial Heap & Fibonacci Heap

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

What is a Binomial Tree?

Recursive Definition

A binomial tree B_0 consists of one node.

For $k \geq 1$, a binomial tree B_k consists of two B_{k-1} trees linked so that the root of one becomes a child of the root of the other.

$$B_{k-1} + B_{k-1} \longrightarrow B_k$$

- The index k is the order of the tree.
- The root of B_k has degree k .
- A binomial tree need not be a binary tree.

Binomial Trees: B_0 , B_1 and B_2

B_0

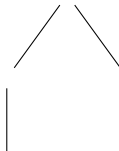
1 node

B_1



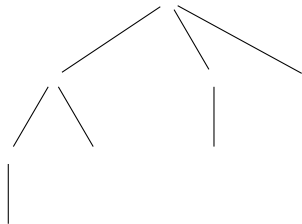
2 nodes

B_2



4 nodes

Binomial Tree B_3



- Nodes: $2^3 = 8$.
- Height: 3 edges.
- Root degree: 3.
- Root children head:

B_2, B_1, B_0 .

Properties of a Binomial Tree

For a binomial tree B_k :

$$\text{Number of nodes} = 2^k$$

$$\text{Height} = k$$

$$\text{Root degree} = k$$

$$\text{Nodes at depth } i = \binom{k}{i}$$

The root's child subtrees have orders:

$$k-1, k-2, \dots, 1, 0.$$

Why the Name Binomial?

For B_3 , the numbers of nodes at successive depths are:

1, 3, 3, 1.

These are binomial coefficients:

$$\binom{3}{0}, \quad \binom{3}{1}, \quad \binom{3}{2}, \quad \binom{3}{3}.$$

Total Number of Nodes

$$\sum_{i=0}^k \binom{k}{i} = (1 + 1)^k = 2^k.$$

What is a Binomial Heap?

Definition

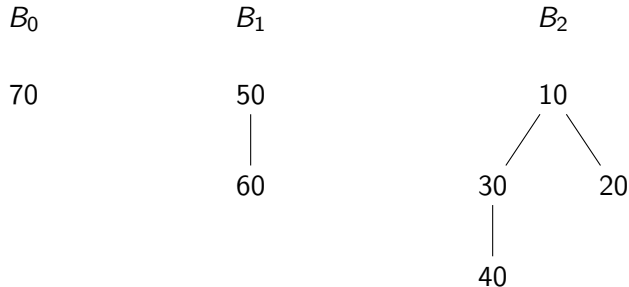
A binomial min-heap is a collection of binomial trees such that:

- ① Every tree obeys the min-heap property.
- ② At most one tree has any particular order.

$$\text{key}(\text{parent}) \leq \text{key}(\text{child}).$$

- Each tree's smallest key is at its root.
- The heap's minimum is among the roots.
- Root lists are commonly stored in increasing degree.

Binomial Heap: Example with Seven Nodes



$$7 = 1 + 2 + 4 = 2^0 + 2^1 + 2^2.$$

Binary Representation Determines Tree Orders

Let:

$$n = \sum_{k \geq 0} b_k 2^k, \quad b_k \in \{0, 1\}.$$

A binomial heap with n nodes contains B_k exactly when $b_k = 1$.

For example:

$$13 = (1101)_2 = 8 + 4 + 1.$$

Therefore:

$$13 \text{ nodes require } B_3, B_2, B_0.$$

Number of Trees

The number of trees equals the number of 1 bits in n , and is at most $\lfloor \log_2 n \rfloor + 1$ for $n \geq 1$.

Binomial-Heap Representation

A typical node stores:

- Key and associated item.
- Degree.
- Parent pointer.
- Pointer to the first child.
- Pointer to the next sibling.

Root List

Roots have null parent pointers. A heap header points to the first root. An optional minimum pointer supports constant-time MINIMUM.

Linking Two Binomial Trees

To link two min-heap-ordered trees of the same order:

- 1 Compare their root keys.
- 2 Keep the smaller-key root as the new root.
- 3 Make the other root its child.
- 4 Increase the new root's degree by one.

$$B_k + B_k \longrightarrow B_{k+1}$$

$$\text{Linking cost} = O(1)$$

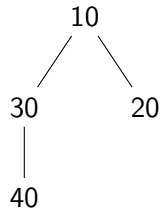
Requirement

Both trees must have the same order.

Linking Example: Two B_1 Trees

10
|
20

30
|
40



$B_1 + B_1$

B_2

Binomial Link Algorithm

```
LINK-ROOTS(x, y)
    // x and y are detached roots of equal degree

    if x.key > y.key
        swap x and y

    y.parent = x
    y.sibling = x.child
    x.child = y
    x.degree = x.degree + 1

    return x
```

Result

The returned root heads a binomial tree of the next order. The min-heap property remains valid.

Melding Binomial Heaps

Melding combines the nodes of two heaps.

- 1 Combine their roots.
- 2 Link roots of equal degree.
- 3 Continue until every root degree is unique.
- 4 Arrange the final roots by increasing degree.

Binary Addition Analogy

Two B_k trees act like two 1 bits at position k . Linking produces a carry into position $k + 1$.

$$\text{Meld time} = O(\log(n_1 + n_2))$$

Meld Example: Degree Carries

Suppose:

$$H_1 : B_0, B_2 \quad (n_1 = 5),$$

$$H_2 : B_0, B_1 \quad (n_2 = 3).$$

The links are:

$$B_0 + B_0 \longrightarrow B_1,$$

$$B_1 + B_1 \longrightarrow B_2,$$

$$B_2 + B_2 \longrightarrow B_3.$$

$$\boxed{5 + 3 = 8 \implies \text{one } B_3}$$

Binomial Meld: Degree-Table Version

```
BINOMIAL-MELD(H1, H2)
    create degree table A, initially all NIL
    R = roots from H1 and H2

    for each root x detached from R
        while A[x.degree] != NIL
            d = x.degree
            y = A[d]
            A[d] = NIL
            x = LINK-ROOTS(x, y)

        A[x.degree] = x

    rebuild root list from occupied A entries
    in increasing degree
    return the resulting heap
```

Use $O(\log(n_1 + n_2 + 1))$ table entries. This is a teaching alternative to the standard list scan.

Why Binomial Meld Takes Logarithmic Time

- Each input heap has $O(\log(n + 1))$ roots.
- Each link decreases the number of trees by one.
- Hence there are $O(\log(n + 1))$ links.
- Scanning and rebuilding the degree table also takes $O(\log(n + 1))$ time.

Here $n = n_1 + n_2$.

$$\text{Total meld time} = O(\log(n + 1))$$

Alternative Implementation

The standard sorted-root-list algorithm avoids the auxiliary degree table and uses $O(1)$ auxiliary space.

Insertion into a Binomial Heap

To insert key x :

- 1 Create a one-node tree B_0 .
- 2 Treat it as a singleton heap.
- 3 Meld it with the existing heap.

$$\text{Worst-case insertion} = O(\log(n + 1))$$

Possible Carry Chain

If the heap contains $B_0, B_1, \dots, B_{k-1}$, one insertion links through all k orders.

Insertion Example: From Seven to Eight Nodes

Start with:

$$B_0(70), \quad B_1(50), \quad B_2(10).$$

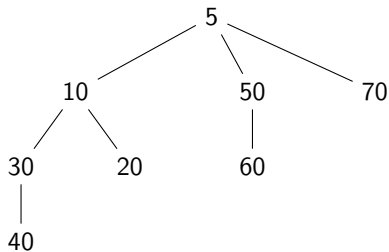
Insert key 5 as a new B_0 .

- ① Link roots 5 and 70 to form B_1 rooted at 5.
- ② Link with the B_1 rooted at 50.
- ③ Link with the B_2 rooted at 10.

$$7 = (111)_2 \quad \longrightarrow \quad 8 = (1000)_2.$$

Three links produce one B_3 .

Insertion Example: Final B_3



Finding the Minimum

The minimum key of each tree is at its root.

Therefore:

- 1 Scan the root list.
- 2 Return the root with the smallest key.

There are at most $\lfloor \log_2 n \rfloor + 1$ roots.

MINIMUM = $O(\log n)$ by root scanning

With a Maintained Minimum Pointer

MINIMUM takes $O(1)$. Other operations must maintain that pointer.

Extract-Min from a Binomial Heap

- 1 Find and remove the minimum root z .
- 2 Separate the child subtrees of z .
- 3 Set their parent pointers to NIL.
- 4 Arrange these roots in increasing degree.
- 5 Meld this child heap with the remaining heap.

Why Reverse the Child List?

With the standard leftmost-child linking convention, a B_k root stores children in decreasing degree: $k-1, \dots, 0$. Reversal gives increasing order.

$$\text{EXTRACT-MIN} = O(\log n)$$

Extract-Min Example

Remove root 5 from the previous B_3 .

Its children become roots:

$B_2(10)$, $B_1(50)$, $B_0(70)$.

Reverse their order:

$B_0(70)$, $B_1(50)$, $B_2(10)$

- Seven nodes remain.
- Their tree orders match $(111)_2$.
- The new minimum is 10.

Binomial Decrease-Key

```
BINOMIAL-DECREASE-KEY(H, x, newKey)
    if newKey > x.key
        report invalid decrease

    x.key = newKey
    y = x
    p = y.parent

    while p != NIL and y.key < p.key
        swap the items stored at y and p
        update item-to-node handles, if maintained
        y = p
        p = y.parent
```

Swap complete items, including their keys. Update an optional minimum pointer after the operation.

Binomial Decrease-Key: Example

Consider the path:

$10 \longrightarrow 30 \longrightarrow 40.$

Decrease key 40 to 2:

$10 \longrightarrow 30 \longrightarrow 2.$

Swap upward:

$10 \longrightarrow 2 \longrightarrow 30,$

$2 \longrightarrow 10 \longrightarrow 30.$

Complexity

A binomial tree has height $O(\log n)$. Therefore decrease-key takes $O(\log n)$ worst-case time.

Deleting a Binomial-Heap Item

For a specified item x :

- ① Decrease its key to a value smaller than every key.
- ② Let it move to a root.
- ③ Apply EXTRACT-MIN.

$$\text{DELETE} = O(\log n)$$

Sentinel Convention

The conceptual value $-\infty$ must identify the target as strictly smallest. Implementations can use a special deletion flag or a suitable tie-breaking mechanism.

Binomial-Heap Complexity

Operation	Worst-Case Time
MAKE-HEAP	$O(1)$
MINIMUM, root scan	$O(\log n)$
MINIMUM, maintained pointer	$O(1)$
INSERT	$O(\log n)$
MELD	$O(\log n)$
EXTRACT-MIN	$O(\log n)$
DECREASE-KEY, given handle	$O(\log n)$
DELETE, given handle	$O(\log n)$

For meld, n denotes the combined size. Use $\log(n + 1)$ when including empty heaps.

Connection to Amortized Analysis

For an optimized insertion that processes only the carry chain, let $t(H)$ be the number of trees. If one insertion performs ℓ links:

$$\text{work} = O(1 + \ell),$$

$$\Delta t = 1 - \ell.$$

Each link reduces the tree count and releases stored credit.

Result

This specialized insertion takes $O(1)$ amortized time. Starting from empty, n such insertions take $O(n)$ total time.

Implementation Matters

A generic meld that scans an entire root list or degree table does not automatically achieve this insertion bound.

Fibonacci Heaps

Deferred Consolidation and Cascading Cuts

What is a Fibonacci Heap?

Definition

A Fibonacci min-heap is a collection of min-heap-ordered rooted trees, represented using linked lists and a pointer to the minimum root.

- Trees may have the same root degree.
- Trees need not retain binomial-tree shapes.
- Insert and meld postpone consolidation.
- Extract-min consolidates equal-degree roots.
- Decrease-key may cut nodes from their parents.

Fibonacci-Heap Representation

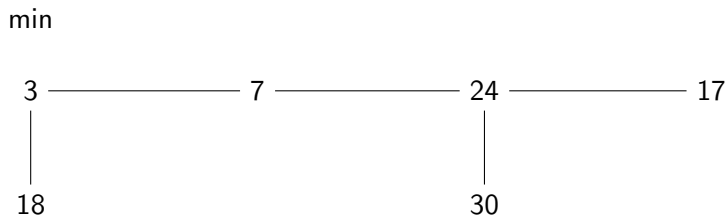
Each node stores:

- Key and associated item.
- Parent pointer and one child pointer.
- Left and right sibling pointers.
- Degree.
- Mark bit.

Heap Header

Stores the minimum-root pointer and total node count. Root lists and child lists use circular doubly linked lists.

Fibonacci Heap: Example



- Root degrees are 1, 0, 1, 0.
- Duplicate root degrees are allowed.

Dashed links indicate the root list. Its circular closure is omitted.

Deferred Consolidation

Binomial Heap

Melding resolves duplicate tree orders immediately.

At most one root has each degree.

Fibonacci Heap

Insertion and melding simply add roots.
Extract-min later resolves duplicate root degrees.

Consequence

A Fibonacci heap can have $\Theta(n)$ roots before extraction. One extract-min call can therefore take $\Theta(n)$ actual time.

Fibonacci-Heap Insertion

```
FIB-INSERT(H, x)
    x.parent = NIL
    x.child = NIL
    x.degree = 0
    x.mark = FALSE

    make x a singleton circular list
    add x to H's root list

    if H.min == NIL or x.key < H.min.key
        H.min = x

    H.n = H.n + 1
```

Actual and amortized time = $O(1)$

Melding Fibonacci Heaps

- 1 Concatenate the two circular root lists.
- 2 Choose the smaller of their minimum roots.
- 3 Add their node counts.

Pointer Operations

Circular doubly linked lists support concatenation with a constant number of pointer updates.

$$\text{Actual and amortized meld time} = O(1)$$

Empty-heap cases return the other heap. No tree linking occurs during meld.

Extract-Min in a Fibonacci Heap

- 1 Let z be the minimum root.
- 2 Promote all children of z to the root list.
- 3 Clear their parent pointers and marks.
- 4 Remove z and decrease the node count.
- 5 Consolidate roots of equal degree.
- 6 Recompute the minimum pointer.

Amortized time = $O(\log n)$

Actual Time

A single extraction can scan many roots and take $O(n)$ time.

Consolidation

Use a degree table:

$A[d]$ = root currently stored with degree d .

For each root:

- 1 Check whether its degree already appears in A .
- 2 If so, link the two roots.
- 3 Keep the smaller key as parent.
- 4 Repeat at the increased degree.

After Consolidation

At most one root has each degree. Later insertions and cuts may introduce duplicates again.

Fibonacci Consolidation Algorithm

CONSOLIDATE(H)

A = degree table, initially all NIL

while H's root list is not empty

 x = detach one root from H's root list

while A[x.degree] != NIL

 d = x.degree

 y = A[d]

 A[d] = NIL

if x.key > y.key

 swap x and y

 attach y as a child of x

 y.parent = x

 y.mark = FALSE

 x.degree = x.degree + 1

Consolidation: Completing the Algorithm

After processing all original roots:

- 1 Rebuild the root list from occupied entries of A .
- 2 Set every root's parent to NIL.
- 3 Clear root marks in this implementation.
- 4 Set the minimum pointer to the smallest root.

Degree-Table Capacity

Allow indices through a valid upper bound on maximum degree. For Fibonacci heaps this requires $O(\log(n + 1))$ entries.

Implementation Detail

Detach roots safely while traversing. Linking changes list membership, so a naive live-list loop can skip nodes.

Extract-Min Example: Remove 3

Start with the illustrated six-node heap:

3(18), 7, 24(30), 17.

Here $a(b)$ denotes a root a with child b .

Remove 3 and promote 18.

Process the resulting roots in this order:

18, 7, 24, 17.

Their degrees are:

0, 0, 1, 0.

Remaining Size

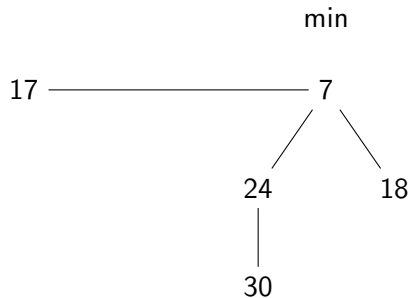
Five nodes remain after extraction.

Extract-Min Example: Consolidation Trace

Root cessed	Pro-	Action
18, degree 0		Store 18 in $A[0]$.
7, degree 0		Link with 18. Root 7 now has degree 1. Store in $A[1]$.
24, degree 1		Link with 7. Root 7 now has degree 2. Store in $A[2]$.
17, degree 0		Store 17 in $A[0]$.

Final roots: 17 of degree 0, 7 of degree 2

Extract-Min Example: Final Heap



New minimum = 7

Decrease-Key in a Fibonacci Heap

To decrease the key of node x :

- 1 Assign the new key.
- 2 Compare it with the parent's key.
- 3 If heap order holds, keep the structure.
- 4 Otherwise cut x and place it in the root list.
- 5 Apply cascading-cut rules to its former parent.
- 6 Update the minimum pointer.

$$\text{Amortized decrease-key time} = O(1)$$

Meaning of the Mark Bit

Marked Nonroot

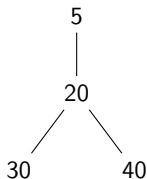
A marked node has lost one child since it most recently became a child of another node.

- First child loss: mark an unmarked nonroot.
- Second child loss: cut that marked node.
- Continue the same rule at its former parent.
- Clear a node's mark when it becomes a root.

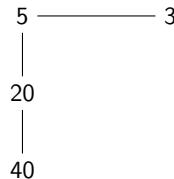
Purpose

The mark records structural history. It does not indicate whether heap order is currently violated.

Decrease-Key Example: First Child Loss



Before: node 20 is unmarked.



Decrease 30 to 3, cut it, and mark 20.

Decrease-Key Example: Cascading Cut

Now decrease key 40 to 2.

- ① Cut node 2 from node 20.
- ② Node 20 was already marked.
- ③ Cut node 20 from node 5.
- ④ Stop at root 5 and clear the mark on 20.

5 ——— 3 ——— 2 ——— 20

All four nodes are now unmarked roots.

Fibonacci Decrease-Key Algorithm

```
FIB-DECREASE-KEY(H, x, newKey)
    if newKey > x.key
        report invalid decrease

    x.key = newKey
    y = x.parent

    if y != NIL and x.key < y.key
        CUT(H, x, y)
        CASCADING-CUT(H, y)

    if x.key < H.min.key
        H.min = x
```

The caller supplies a valid handle to x .

Cut Algorithm

```
CUT(H, x, y)
    remove x from y's child list

    update y.child if necessary
    y.degree = y.degree - 1

    add x to H's root list
    x.parent = NIL
    x.mark = FALSE
```

Constant-Time Cut

Doubly linked sibling lists allow removal of a known node without scanning all its siblings.

One cut = $O(1)$ actual time

Cascading-Cut Algorithm

```
CASCADING-CUT(H, y)
    z = y.parent

    if z != NIL
        if y.mark == FALSE
            y.mark = TRUE
        else
            CUT(H, y, z)
            CASCADING-CUT(H, z)
```

Stopping Condition

Stop at an unmarked nonroot after marking it, or stop on reaching a root.

Deleting a Fibonacci-Heap Node

For a node x :

- 1 Decrease its key to a conceptual value $-\infty$.
- 2 Ensure it becomes the designated minimum.
- 3 Apply EXTRACT-MIN.

$$O(1) \text{ amortized} + O(\log n) \text{ amortized} = \boxed{O(\log n) \text{ amortized}}.$$

Target Identity

The sentinel or tie-breaking rule must select the specified node, even when ordinary keys can be equal.

Why the Name Fibonacci Heap?

Cuts change tree shapes, so a degree- d node need not head an exact 2^d -node binomial tree. However, the marking rule guarantees:

$$\text{Subtree size of a degree-}d \text{ node} \geq F_{d+2}.$$

Here:

$$F_0 = 0, \quad F_1 = 1, \quad F_{i+2} = F_{i+1} + F_i.$$

Consequence

Fibonacci numbers grow exponentially. Therefore maximum node degree is $O(\log n)$.

Degree Bound: Why Cuts Must Be Controlled

Order a node's current children by the time they were linked:

$$y_1, y_2, \dots, y_d.$$

When y_i was linked:

- Its new parent already had at least $i - 1$ children.
- Equal-degree linking gave y_i at least $i - 1$ children.
- It can lose at most one child while remaining attached.

Thus:

$$\deg(y_i) \geq \max(0, i - 2).$$

This lower bound yields the Fibonacci subtree-size recurrence.

Maximum Degree is Logarithmic

Let:

$$\varphi = \frac{1 + \sqrt{5}}{2}.$$

Since:

$$F_{d+2} \geq \varphi^d,$$

a node of degree d satisfies:

$$n \geq F_{d+2} \geq \varphi^d.$$

Therefore:

$$d \leq \log_{\varphi} n = O(\log n).$$

Degree Versus Height

This bounds the number of children. It does not guarantee logarithmic tree height.

Fibonacci-Heap Potential Function

Let:

$t(H)$ = number of roots,

$m(H)$ = number of marked nodes.

Use the potential:

$$\Phi(H) = t(H) + 2m(H).$$

- Root credit pays for future consolidation.
- Mark credit pays for cascading cuts.
- Empty heaps have zero potential.
- Potential is always nonnegative.

A fixed constant scaling can cover the chosen primitive-operation cost model.

Potential Analysis: Insert and Meld

Insertion

Insertion creates one root:

$$\Delta t = 1, \quad \Delta m = 0.$$

Actual work and potential increase are both constant.

$$\text{Amortized insertion} = O(1)$$

Meld

Root and mark counts add across the two heaps. Relative to their combined old potential, $\Delta\Phi = 0$.

$$\text{Amortized meld} = O(1)$$

Potential Analysis: Decrease-Key

Suppose decrease-key performs $c \geq 1$ cuts.

- Each cut creates one root: $\Delta t = c$.
- At least $c - 1$ previously marked nodes lose their marks.
- At most one previously unmarked node becomes marked.

Therefore:

$$\Delta m \leq 1 - (c - 1) = 2 - c,$$

$$\Delta \Phi \leq c + 2(2 - c) = 4 - c.$$

Actual work is $O(c + 1)$. With a suitable constant scaling of potential:

$$\text{Amortized decrease-key} = O(1).$$

Potential Analysis: Extract-Min

Let t be the old number of roots and $D(n) = O(\log n)$ the maximum degree.

- Promote at most $D(n)$ children.
- Scan roots and consolidate in $O(t + D(n))$ actual time.
- At most $D(n) + 1$ roots remain.
- No new marked nodes are created.

Thus:

$$\Delta\Phi \leq D(n) + 1 - t.$$

The decrease in root potential pays for the large root scan.

$$\text{Amortized extract-min} = O(\log n)$$

Fibonacci Heap: Actual and Amortized Bounds

Operation	Worst-Case Actual	Amortized
MAKE-HEAP	$O(1)$	$O(1)$
MINIMUM	$O(1)$	$O(1)$
INSERT	$O(1)$	$O(1)$
MELD	$O(1)$	$O(1)$
EXTRACT-MIN	$O(n)$	$O(\log n)$
DECREASE-KEY	$O(n)$	$O(1)$
DELETE	$O(n)$	$O(\log n)$

Important

A long cascading cut may take linear actual time. Constant amortized time is a sequence guarantee.

Binary, Binomial and Fibonacci Heaps

Operation	Binary	Binomial	Fibonacci
MINIMUM	$O(1)$	$O(1)^*$	$O(1)$
INSERT	$O(\log n)$	$O(\log n)$	$O(1)$
MELD	$O(n)$	$O(\log n)$	$O(1)$
EXTRACT-MIN	$O(\log n)$	$O(\log n)$	$O(\log n)$
DECREASE-KEY	$O(\log n)$	$O(\log n)$	$O(1)$
DELETE	$O(\log n)$	$O(\log n)$	$O(\log n)$

Binary and binomial columns give worst-case bounds. Fibonacci gives amortized bounds. *Binomial MINIMUM assumes a maintained minimum pointer. Meld uses combined size n . Updates assume a node handle or array index.

Building Heaps

Heap	Construction Method	Time
Binary	Bottom-up heapify	$O(n)$
Binomial	Optimized insertions that process only carry chains	$O(n)$ total
Fibonacci	Insert each item as a singleton root	$O(n)$

Fibonacci-Heap Construction

The resulting heap may have n roots. It is valid even before consolidation.

Applications to Graph Algorithms

Dijkstra's algorithm and Prim's algorithm may perform:

$O(V)$ insertions and extractions,

$O(E)$ decrease-key operations.

Using Fibonacci heaps:

$$O(V \log V) + O(E) = \boxed{O(E + V \log V)}.$$

Using binary heaps:

$$\boxed{O((V + E) \log V)}.$$

Assumes adjacency-list implementations with handles. Dijkstra's algorithm requires nonnegative edge weights.

Practical Trade-Offs

- Binary heaps use compact arrays and have good locality.
- Binomial heaps offer logarithmic worst-case melding.
- Fibonacci heaps provide strong amortized bounds for frequent decrease-key operations.
- Fibonacci heaps require more pointers and bookkeeping.
- Actual performance depends on the workload and implementation.

Choice of Structure

Asymptotic bounds, memory overhead and implementation complexity all matter.

Worked Problem 1: Binomial Tree Orders

Question

Which trees occur in a binomial heap containing 29 nodes?

$$29 = 16 + 8 + 4 + 1 = (11101)_2.$$

Therefore:

$$B_4, B_3, B_2, B_0.$$

- Number of trees: 4.
- Largest root degree: 4.
- Maximum tree height: 4.

Worked Problem 2: Insertion Carries

Question

How many links occur when inserting into a binomial heap with 15 nodes?

$$15 = (1111)_2.$$

The heap contains:

$$B_0, B_1, B_2, B_3.$$

One insertion produces:

$$16 = (10000)_2.$$

Four links produce one B_4 .

Worked Problem 3: Fibonacci Potential

Question

A Fibonacci heap has 8 roots and 3 marked nodes. Find its potential.

$$\Phi(H) = t(H) + 2m(H).$$

$$\Phi(H) = 8 + 2(3) = \boxed{14}.$$

After inserting an unmarked singleton:

$$t' = 9, \quad m' = 3,$$

$$\boxed{\Phi(H') = 15}.$$

Worked Problem 4: Cascading Cuts

Question

A decrease-key performs 4 cuts. Bound the change in potential.

There are 4 new roots:

$$\Delta t = 4.$$

At least 3 marked nodes become unmarked, and at most one new mark appears:

$$\Delta m \leq 1 - 3 = -2.$$

Therefore:

$$\Delta \Phi \leq 4 + 2(-2) = \boxed{0}.$$

Stored credit pays for the cascading work.

Common Mistakes

- Assuming a binomial tree is a binary tree.
- Allowing duplicate orders in a finished binomial heap.
- Assuming Fibonacci trees always retain binomial shapes.
- Consolidating during every Fibonacci insertion.
- Treating a mark as a heap-order violation.
- Confusing maximum degree with tree height.
- Reporting amortized bounds as per-call worst cases.

Practice Questions

- 1 List the binomial trees required for 42 nodes.
- 2 How many nodes are in B_5 ?
- 3 How many nodes occur at depth 2 in B_5 ?
- 4 How many links follow insertion into a 31-node binomial heap?
- 5 Find the Fibonacci potential for 6 roots and 4 marked nodes.
- 6 Why can one Fibonacci extract-min take linear time?

Practice Questions: Answers

- ① $42 = 32 + 8 + 2$, so B_5, B_3, B_1 .
- ② $2^5 = \boxed{32}$ nodes.
- ③ $\binom{5}{2} = \boxed{10}$ nodes.
- ④ $31 = (11111)_2$: $\boxed{5}$ links.
- ⑤ $\Phi = 6 + 2(4) = \boxed{14}$.
- ⑥ The root list can contain $\Theta(n)$ roots that must be scanned and consolidated.

Quick Revision

Concept	Key Point
Binomial tree B_k	2^k nodes, height k
Binomial heap	At most one tree per order
Binomial linking	Equal orders, smaller root becomes parent
Fibonacci root list	Duplicate degrees allowed
Fibonacci insertion	Add an unmarked singleton root
Consolidation	Link roots of equal degree
Mark bit	Records one child loss while a nonroot
Cascading cut	Cut an already marked parent after another loss
Potential	$\Phi(H) = t(H) + 2m(H)$

Summary

- Binomial heaps use a strict collection of binomial trees.
- Their tree orders correspond to the binary form of n .
- Fibonacci heaps defer consolidation until extract-min.
- Marking and cascading cuts preserve degree bounds.
- Potential analysis pays for deferred work.

Fibonacci decrease-key: $O(1)$ amortized

Fibonacci extract-min: $O(\log n)$ amortized

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein. *Introduction to Algorithms*, 2nd edition. MIT Press, 2001. Chapters 19 and 20: Binomial Heaps and Fibonacci Heaps.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein. *Introduction to Algorithms*, 3rd edition. MIT Press, 2009. Chapter 19: Fibonacci Heaps.
- MIT OpenCourseWare, 6.854 Advanced Algorithms, Fall 2005. Lecture notes on Fibonacci heaps.

Thank You